EX.NO: 01
DATE:

CANDIDATE ELIMINATION ALGORITHM

AIM:

To implement the **Candidate Elimination Algorithm** for learning a target concept and to determine the **Specific Boundary (S)** and **General Boundary (G)** of the version space using given training examples.

CANDIDATE ELIMINATION ALGORITHM (CONCEPT)

Candidate Elimination is a **concept learning algorithm** that maintains a **version space**, which is the set of all hypotheses consistent with the training data.

It maintains two boundaries:

Specific Boundary (S):

The most specific hypothesis that fits all positive examples.

General Boundary (G):

The most general hypotheses that do not cover any negative examples.

The algorithm iteratively updates S and G as new training examples are processed.

ALGORITHM (STEPS)

1. Initialize:
   - Specific hypothesis **S** with the most specific values (Ø).
   - General hypothesis **G** with the most general values (?).
2. For each training example:
   - If the example is **positive**:
     Generalize S minimally to include the example.
     Remove hypotheses from G that do not cover the example.
   - If the example is **negative**:
     Specialize G minimally to exclude the example.
     Remove hypotheses that are inconsistent or redundant.
3. Prune G to keep only **maximally general hypotheses**.
4. Continue until all examples are processed.
5. The final S and G define the **version space**.

Intelligent System for Remote Work Approval

Problem Overview

A startup wants to decide whether an employee should be approved for **Remote Work** based on the following attributes:

1. **Role** – Technical / Non-Technical
2. **Experience** – Junior / Senior
3. **Performance** – Average / Excellent
4. **Internet Quality** – Poor / Good
5. **Work Location** – Urban / Rural

Target Concept: RemoteWorkApproval = Yes / No

Training Examples

No	Role	Experience	Performance	Internet	Location	Approval
1	Technical	Senior	Excellent	Good	Urban	Yes
2	Technical	Junior	Excellent	Good	Urban	Yes
3	Non-Technical	Junior	Average	Poor	Rural	No
4	Technical	Senior	Average	Good	Rural	No
5	Technical	Senior	Excellent	Good	Rural	Yes

Q1. Initialization

a) Initial Hypotheses
Most Specific Hypothesis (S_0):
[⟨ Ø, Ø, Ø, Ø, Ø ⟩]
Most General Hypothesis (G_0):
[⟨ ?, ?, ?, ?, ? ⟩]

b) Why These Are Chosen
S_0 represents *no knowledge* and matches **no instances**.
G_0 represents *maximum generality* and matches **all instances**.
Candidate Elimination shrinks the version space from both ends using data.

Q2. Step-by-Step Learning (Candidate Elimination Algorithm)

Example 1 (Positive)
⟨Technical, Senior, Excellent, Good, Urban⟩ → Yes
Update S to match the example
G remains unchanged
S: ⟨Technical, Senior, Excellent, Good, Urban⟩
G: ⟨?, ?, ?, ?, ?⟩

Example 2 (Positive)
⟨Technical, Junior, Excellent, Good, Urban⟩ → Yes
Generalize S where values differ
S: ⟨Technical, ?, Excellent, Good, Urban⟩
G: ⟨?, ?, ?, ?, ?⟩

Example 3 (Negative)
⟨Non-Technical, Junior, Average, Poor, Rural⟩ → No
S unchanged (does not cover negative example)
Specialize G to exclude this negative example
G:
⟨Technical, ?, ?, ?, ?⟩
⟨?, ?, Excellent, ?, ?⟩
⟨?, ?, ?, Good, ?⟩
⟨?, ?, ?, ?, Urban⟩
S: ⟨Technical, ?, Excellent, Good, Urban⟩

Example 4 (Negative)
⟨Technical, Senior, Average, Good, Rural⟩ → No
Remove hypotheses in G that cover this negative example
Remaining G:
⟨?, ?, Excellent, ?, ?⟩
⟨?, ?, ?, ?, Urban⟩
S: unchanged

Example 5 (Positive)
⟨Technical, Senior, Excellent, Good, Rural⟩ → Yes
Generalize S minimally

Remove inconsistent hypotheses from G
Final S: ⟨Technical, ?, Excellent, Good, ?⟩
Final G: ⟨?, ?, Excellent, ?, ?⟩

Q3. Version Space Analysis

a) Meaning of Final Version Space
The final version space contains **all hypotheses** that:
Approve remote work **only** for employees with *Excellent performance*
Are consistent with all given training examples

b) Two Valid Hypotheses in the Version Space
⟨Technical, ?, Excellent, Good, ?⟩
⟨?, ?, Excellent, ?, ?⟩

Q4. Prediction
Test Instance
⟨Technical, Senior, Excellent, Good, Urban⟩
Covered by **S** and **G**
Prediction
✅ **Remote Work Approved (Yes)**
Justification:
The instance satisfies the necessary condition *Performance* = *Excellent* and is consistent with both boundaries.

Q5. Conceptual Understanding

a) Effect of Contradictory Examples
Version space may become **empty**
Indicates inconsistent or incorrect data

b) Handling Noisy Data
Candidate Elimination **cannot handle noise**
Requires all examples to be perfectly consistent

Q6. Critical Thinking

a) Real-World Limitation
Requires **complete and noise-free data**, which is unrealistic in HR decisions

b) Alternative Approach
Decision Trees (ID3 / C4.5) or **Naïve Bayes**
These handle noise and uncertainty better

Q7. Extension: Adding a New Attribute
New Attribute
Team Dependency – Low / High
Impact on Hypothesis Representation
Hypothesis size increases from 5 to **6 attributes**

Example:
⟨Role, Experience, Performance, Internet, Location, Team Dependency⟩

Impact on Version Space
Version space increases exponentially
More hypotheses → higher computational complexity

CODE:

```
data=[(["Technical","Senior","Excellent","Good","Urban"],"Yes"),(["Technical","Junior","Excellen
t","Good","Urban"],"Yes"),(["Non-
Technical","Junior","Average","Poor","Rural"],"No"),(["Technical","Senior","Average","Good","R
ural"],"No"),(["Technical","Senior","Excellent","Good","Rural"],"Yes")]
num_attributes=5
S=["Ø"]*num_attributes
G=[["?"]*num_attributes]
def is_consistent(hypothesis,example):
    for h,e in zip(hypothesis,example):
        if h!="?" and h!=e:
            return False
    return True
def more_general(h1,h2):
    for x,y in zip(h1,h2):
        if x!="?" and (y=="?" or x!=y):
            return False
    return True
def prune_G(G):
    pruned=[]
    for g in G:
        if not any(more_general(other,g) and other!=g for other in G):
            pruned.append(g)
    return pruned
for index,(example,label) in enumerate(data,start=1):
    print(f"\nExample {index}: {example} -> {label}")
    if label=="Yes":
        for i in range(num_attributes):
            if S[i]=="Ø":
                S[i]=example[i]
            elif S[i]!=example[i]:
                S[i]="?"
        G=[g for g in G if is_consistent(g,example)]
    else:
        new_G=[]
        for g in G:
            if is_consistent(g,example):
                for i in range(num_attributes):
                    if g[i]=="?":
                        if S[i]!="?" and S[i]!=example[i]:
                            new_hypothesis=g.copy()
                            new_hypothesis[i]=S[i]
                            new_G.append(new_hypothesis)
```

```
            else:
                new_G.append(g)
        G=new_G
    G=prune_G(G)
    print("Specific Boundary S:",S)
    print("General Boundary G:",G)
print("\nFINAL RESULT")
print("Final Specific Hypothesis (S):",S)
print("Final General Hypotheses (G):",G)
```

Observation	15	
Implementation	35	
Output	10	
Viva	10	
Record	05	
Total	75	

RESULT:

Thus implement the **Candidate Elimination Algorithm** for learning a target concept and to determine the **Specific Boundary (S)** and **General Boundary (G)** of the version space and uploaded in Github

Github link:
VarshiniRamesh2005/machine-learning

OUTPUT:

```
Example 1: ['Technical', 'Senior', 'Excellent', 'Good', 'Urban'] -> Yes
Specific Boundary S: ['Technical', 'Senior', 'Excellent', 'Good', 'Urban']
General Boundary G: [['?', '?', '?', '?', '?']]

Example 2: ['Technical', 'Junior', 'Excellent', 'Good', 'Urban'] -> Yes
Specific Boundary S: ['Technical', '?', 'Excellent', 'Good', 'Urban']
General Boundary G: [['?', '?', '?', '?', '?']]

Example 3: ['Non-Technical', 'Junior', 'Average', 'Poor', 'Rural'] -> No
Specific Boundary S: ['Technical', '?', 'Excellent', 'Good', 'Urban']
General Boundary G: [['Technical', '?', '?', '?', '?'], ['?', '?', 'Excellent', '?', '?'], ['?', '?', '?', 'Good', '?'], ['?', '?', '?', '?', 'Urban']]

Example 4: ['Technical', 'Senior', 'Average', 'Good', 'Rural'] -> No
Specific Boundary S: ['Technical', '?', 'Excellent', 'Good', 'Urban']
General Boundary G: [['?', '?', 'Excellent', '?', '?'], ['?', '?', '?', '?', 'Urban']]

Example 5: ['Technical', 'Senior', 'Excellent', 'Good', 'Rural'] -> Yes
Specific Boundary S: ['Technical', '?', 'Excellent', 'Good', '?']
General Boundary G: [['?', '?', 'Excellent', '?', '?']]

✅ FINAL RESULT
Final Specific Hypothesis (S): ['Technical', '?', 'Excellent', 'Good', '?']
Final General Hypotheses (G): [['?', '?', 'Excellent', '?', '?']]
```